



University of Engineering and Technology, Lahore

Department of Computer Science

Project Report

Team Members

Name	Roll Number	Email
Muhammad Sheharyar	2025-CS-154	msheharyar1885@gmail.com
Muhammad Adnan Saleem	2024-CS-171	adnancs171@gmail.com

Session: Fall'25 Morning

Section: C

Semester: 1st

Course: CSC101 - Discrete Mathematics

Teacher: Mr. Waqas Ali

January 1, 2026

Table of Contents

Table of Contents	3
Project Report	4
1. Project Title	4
2. Chosen Track	4
3. Overview	4
4. Summary	4
5. Technology Stack & Libraries Utilized	5
6. List of Features	5
7. Mathematical Foundations	5
8. Examination of Key Functions/Methods	5
9. Limitations	10
10. User Interface	10
10.1 Interface Overview	10
10.2 Screen-by-Screen Documentation	10
10.3 Sample Test Cases	14
11. Future Improvements	14
12. Conclusion	14
13. References & Learning Sources	15

Project Report

Submission Deadline: January 6, 2026

1. Project Title

“Function Properties Checker”

2. Chosen Track

State which research track your project belongs to. This helps contextualize your work within the course curriculum and clarifies the primary discrete mathematics concepts you are applying.

- Track 1: Foundations of Logic
- ✓ Track 2: Discrete Structures
- Track 3: Algorithm Design & Analysis
- Track 4: Graph & Tree Algorithms
- Track 5: Number Theory Applications
- Track 6: Counting & Combinatorics

3. Overview

*The Project “**Function Properties Checker**” studies mathematical functions. It helps identify whether a function is bijective, injective and surjective. It also finds inverse of a function only if they meet the condition that is bijective. This project computes the composition of a given function. It also computes the encryption and decryption of a given function. It solves the problem of finding the injectivity, surjectivity and bijectivity of a function, and also encrypt and decrypt the given message.*

4. Summary

This project checks the important properties of mathematical functions, which is the main discrete structures in Discrete Mathematics. This Program takes the input as domain and co-domain of function. And checks the Important properties of function like injectivity, surjectivity and bijectivity. Secondly, it computes the inverse of a function only if given function is bijective. Thirdly, it computes the composition of a function. Fourthly, it also maps the function for visualization so that to understand and learn easily. Fifthly, it also uses the cryptography concept to make the message encrypt or decrypt. To encrypt the message, it asks for key to encrypt the message and it also decrypt the message by asking key. So, it uses both the ceaser cipher and shift cipher to encrypt and decrypt the message by taking the key from user. This project solves the problem of function properties like bijectivity, surjectivity and injectivity. This finds the inverse of function also which sometimes find it difficult for us to calculate. This also finds the composition of a function. All these are basic concepts in old school and we are studying these concepts in course of Discrete Mathematics. But sometimes, we are confused to solve these types of problem. To solve these problems, we understanding the use of functions in our real-world life and learn how to it participate in real-world. Methodologies we are implemented are basic concepts, how to implement in program and concept of cryptography especially ceaser cipher and shift cipher. We achieve how to implement these concepts in editor and how function behaves when putting different inputs. This added our insight towards these types of structures to learn and understand and implement in real-world. This project matters in the context of discrete mathematics because it plays a central and core role in solving real-world applications.

5. Technology Stack & Libraries Utilized

- **Programming Languages:** C++
- **Libraries:** iostream, string, limit
- **Development Environment:** The IDE that we used is Visual Studio Code (VS CODE). The reason behind using VS CODE as IDE is that it is a well-structured and easy to use.

Library (string) is used for string functionalities like string. Length () function etc. And limit library is used to clears input buffer safely before getline () and to ensures all leftovers characters are removed.

6. List of Features

*The **Function Properties Checker** project provides several functional features organized by mathematical analysis are:*

- **Function Property Checker:** Determines whether a given function is injective, surjective, or bijective by analyzing mappings between domain and codomain. [Core Feature]
- **Inverse Function Finder:** Computes the inverse of a function when it exists by reversing valid mappings and verifying bijectivity. [Core Feature]
- **Function Composition:** Calculates the composition of two functions. [Core Feature]
- **Cryptography Module:** Encrypts and decrypts messages using basic mathematical transformations to demonstrate practical applications of functions. [Optional Feature]

7. Mathematical Foundations

The project is based on the following mathematical concepts:

- **Relevant Concepts:**

Relevant Concepts are: **1. Functions and Relations**

2. Injective, Surjective, and Bijective Mappings

3. Inverse Functions

4. Function Composition

5. Cryptography

- **Formal Definitions:**
 1. A function $f: A \rightarrow B$ is injective if: $f(x_1) = f(x_2)$ only if $x_1 = x_2$.
 2. A function is surjective if every element of the codomain has a pre-image.
 3. A function is bijective if it is both injective and surjective.
 4. The inverse function exists only if the function is bijective.
- **Proofs & Correctness:** The correctness of the algorithms relies on checking mathematical definitions. This could include:
 - Injectivity is verified by ensuring no two domain elements maps to same co-domain.
 - Surjectivity is confirmed by covering the entire codomain.
 - Inverse of a function compute only if the function is bijective (Injective and Subjective).
- **Illustrative Example:** Example of function properties checker is:

If

Domain={1,2,3}

Codomain={a,b,c}

Mappings={(1,a),(2,b),(3,c)}

Then the function is bijective and has an inverse.

8. Examination of Key Functions/Methods

For each key function, the proper classification of function includes:

1)

- **Function Name & Signature:** Analyze the function
- **Purpose:** Determines whether a function is bijective, surjective and bijective or not a function.
- **Algorithm Description:** Compares all output values to detect duplicates.
- **Implementation Details:** This function checks whether a given function is **injective, surjective or bijective**. Injectivity, Surjectivity and Bijectivity is a fundamental property in discrete mathematics and is essential for determining whether a function can have an inverse. This function plays a key role in classifying functions within the project.
- **Code Snippet:** C++ code

// ANALYZIE THE FUNCTION by checking function properties like injectivity, surjectivity and bijectivity of a function

```
void analyzeFunction()
{
    bool isInjective = true;
    bool isSurjective = true;
    // Injective check
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (image[i] == image[j])
            {
                isInjective = false;
            }
        }
    }
    // Surjective check
    for (int i = 0; i < m; i++)
    {
        bool found = false;
        for (int j = 0; j < n; j++)
        {
            if (codomain[i] == image[j])
            {
                found = true;
                break;
            }
        }
        if (!found)
        {
            isSurjective = false;
        }
    }
    cout << "Function analysis result:" << endl;
    if (isInjective)
        cout << "Function is injective." << endl;
    else
```

```

    cout << "Function is not injective." << endl;
if (isSurjective)
    cout << "Function is surjective." << endl;
else
    cout << "Function is not surjective." << endl;
if (isInjective && isSurjective)
    cout << "Function is bijective." << endl;
else
    cout << "Function is not bijective." << endl;
}

```

II)

- **Function Name & Signature:** Inverse of Function
- **Purpose:** Determines the inverse of a function.
- **Algorithm Description:** Inverses the input to output if satisfy the condition of bijective.
- **Implementation Details:** This function checks whether a given function is **bijective** after checking this and compute inverse of a function. If the given function is meeting this condition, then it finds the inverse of a function.
- **Code Snippet:** C++ code

```

// INVERSE FUNCTION
void computeInverseFunction()
{
    bool isInjective = true;
    bool isSurjective = true;
    // Check Injective
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (image[i] == image[j])
            {
                isInjective = false;
            }
        }
    }
    if (!isInjective)
        break;
}
// Check Surjective
for (int i = 0; i < m; i++)
{
    bool found = false;
    for (int j = 0; j < n; j++)
    {
        if (codomain[i] == image[j])
        {
            found = true;
            break;
        }
    }
}

```

```

    }
}
if (!found)
{
    isSurjective = false;
    break;
}
}
// Only compute inverse if bijective
if (isInjective && isSurjective)
{
    cout << "Inverse function:" << endl;
    for (int i = 0; i < n; i++)
    {
        cout << "f inverse of " << image[i] << " is " << domain[i] << endl;
    }
}
else
{
    cout << "Inverse function does not exist (function is not bijective)." << endl;
}
}

```

III)

- **Function Name & Signature:** Composition of a function
- **Purpose:** Determines the composition of the function.
- **Algorithm Description:** Composite the function with value of domain.
- **Implementation Details:** It combines two functions by passing the output of one as the input to another. This helps build complex operations from simpler functions.
- **Code Snippet:** C++ code

```

// compute function
void computeFunction()
{
    int x;
    cout << "Enter value to compute f(x): ";
    cin >> x;
    for (int i = 0; i < n; i++)
    {
        if (domain[i] == x)
        {
            cout << "f(" << x << ") = " << image[i] << endl;
            return;
        }
    }
    cout << "Input not in domain." << endl;
}

```


IV)

- **Function Name & Signature:** Encryption of a function.
- **Purpose:** It encrypts the message by using cryptography.
- **Algorithm Description:** Ceaser cipher and shift cipher.
- **Implementation Details:** This function transforms the plain text into cipher text by times as user inputs the key and the encrypted message is shown.
- **Code Snippet:** C++ code

```
// // encrypt Messages
string encryptMessage(string message, int key)
{
    string result = "";
    for (int i = 0; i < message.length(); i++)
    {
        char c = message[i];
        if (c >= 'A' && c <= 'Z')
            result += char((c - 'A' + key) % 26 + 'A');
        else if (c >= 'a' && c <= 'z')
            result += char((c - 'a' + key) % 26 + 'a');
        else
            result += c;
    }
    return result;
}
```

V)

- **Function Name & Signature:** Decryption of a function.
- **Purpose:** It decrypts the message by using cryptography.
- **Algorithm Description:** Ceaser cipher and shift cipher.
- **Implementation Details:** This function transforms the cipher text into plain text by times as user inputs the key and the decrypt message is shown.
- **Code Snippet:** C++ code

```
// decryption of a function
string decryptMessage(string message, int key)
{
    string result = "";
    for (int i = 0; i < message.length(); i++)
    {
        char c = message[i];
        if (c >= 'A' && c <= 'Z')
            result += char((c - 'A' - key + 26) % 26 + 'A');
        else if (c >= 'a' && c <= 'z')
            result += char((c - 'a' - key + 26) % 26 + 'a');
        else
            result += c;
    }
    return result;
}
```

}

9. Limitations

Critically evaluate the limitations of your project. Be honest and thorough. Discuss:

- **Scope Limitations:** It handles only finite and discrete functions and no support for continuous or large-functions.
- **Performance Constraints:** Its maximum range is 1000 for entering data and after this limit no input will be taken.
- **Algorithmic Limitations:** Cryptography implementation is basic and not secure for higher level cryptography.
- **Design Trade-offs:** Code readability was prioritized over advanced optimization.
- **Known Issues:** Limited error message show when entering unknown data and when user puts string whereas entering integer type, then program is out of control. In future, we solve also this problem.

10. User Interface

*The project uses a **Command Line Interface (CLI)** designed for simplicity and clarity. Users interact through menu-driven options.*

```
=====
                        FUNCTION PROPERTIES CHECKER
=====
1. Input Function
2. Analyze Function
3. Compute Inverse Function
4. Compute Function Value
5. Encrypt Message
6. Decrypt Message
7. Exit
Enter choice: █
```

10.1 Interface Overview

The user interface consists of menu that is displayed to the user to choose options. The program asks for a user to enter number of elements for domain and enter elements and then asks for co-domain. Then asks for options to evaluate this operation. In encryption and decryption, it asks for message and key and give the user encrypted or decrypted message.

10.2 Screen-by-Screen Documentation

For each major screen or view in your interface:

- **Main Menu**
- **Screenshot:** A clear screenshot

```

=====
FUNCTION ANALYSIS SYSTEM
=====
1. Input Function
2. Analyze Function
3. Compute Inverse Function
4. Compute Function Value
5. Encrypt Message
6. Decrypt Message
7. Exit
Enter choice: █

```

- **Purpose:** User have options to select one for desired option
- **User Actions:** List of Available options as shown.
- **Input Description:** User have to enter the options as given
- **Output Description:** After selecting options, the desired operations are performed.

- **1. Input Function**
- **Screenshot:** A clear screenshot

```

Enter choice: 1
Enter number of domain elements: 2
Domain element 1: 4
Domain element 2: 5
Enter number of codomain elements: 2
Codomain element 1: 3
Codomain element 2: 6
Enter function values f(x):
f(4) = 3
f(5) = 6
Real function entered successfully.
Function in set notation:
f = { (4, 3) (5, 6) }
Step by step mapping:
4 maps to 3
5 maps to 6

```

- **Purpose:** User enters the domain and co-domain elements
- **User Actions:** User enter 1.
- **Input Description:** Elements of domain and co-domain.
- **Output Description:** Real function is entered successfully.

- **2. Analyze Function**
- **Screenshot:** A clear screenshot

```
=====
FUNCTION ANALYSIS SYSTEM
=====
1. Input Function
2. Analyze Function
3. Compute Inverse Function
4. Compute Function Value
5. Encrypt Message
6. Decrypt Message
7. Exit
Enter choice: 2
Function analysis result:
Function is injective.
Function is surjective.
Function is bijective.
```

- **Purpose:** To analyze the function properties.
- **User Actions:** User enter 2.
- **Input Description:** User enter 2.
- **Output Description:** Function analysis result is shown.

- **3. Compute Inverse Function**
- **Screenshot:** A clear screenshot

```
=====
FUNCTION ANALYSIS SYSTEM
=====
1. Input Function
2. Analyze Function
3. Compute Inverse Function
4. Compute Function Value
5. Encrypt Message
6. Decrypt Message
7. Exit
Enter choice: 3
Inverse function:
f inverse of 3 is 4
f inverse of 6 is 5
```

- **Purpose:** To find the inverse of a function.
- **User Actions:** User enter 3.
- **Input Description:** User enter 3.
- **Output Description:** Inverse of a function is shown.

- **4. Compute Function**
- **Screenshot:** A clear screenshot

```

=====
FUNCTION ANALYSIS SYSTEM
=====
1. Input Function
2. Analyze Function
3. Compute Inverse Function
4. Compute Function Value
5. Encrypt Message
6. Decrypt Message
7. Exit
Enter choice: 4
Enter value to compute f(x): 4
f(4) = 3

```

- **Purpose:** To compute composition of a function.
- **User Actions:** User enters 4.
- **Input Description:** User enters 4.
- **Output Description:** Compute functional value.

- **5. Encryption of a message**
- **Screenshot:** A clear screenshot

```

=====
FUNCTION ANALYSIS SYSTEM
=====
1. Input Function
2. Analyze Function
3. Compute Inverse Function
4. Compute Function Value
5. Encrypt Message
6. Decrypt Message
7. Exit
Enter choice: 5
Enter message: Today is Tuesday
Enter key (0 to 25): 3
Encrypted message: Wrgdb lv wxhvgdb

```

- **Purpose:** To convert the plain text into cipher text.
- **User Actions:** User enters 5
- **Input Description:** User enters 5
- **Output Description:** Encrypted message is shown.

- **6. Decryption of a message**
- **Screenshot:** A clear screenshot

```

=====
FUNCTION ANALYSIS SYSTEM
=====
1. Input Function
2. Analyze Function
3. Compute Inverse Function
4. Compute Function Value
5. Encrypt Message
6. Decrypt Message
7. Exit
Enter choice: 6
Enter encrypted message: wrgdb lv wxhvgdb
Enter key (0 to 25): 3
Decrypted message: Today is Tuesday

```

- **Purpose:** To convert the cipher text into plain text.
- **User Actions:** User enters 6
- **Input Description:** User enters 6
- **Output Description:** Decrypted message is shown.

10.3 Sample Test Cases

Provide 2–3 concrete examples demonstrating typical usage of your interface. For each test case:

- **Test Case Name:** Check function properties
- **Input:** Input (No. of Domain=2 {3,6} and Co-domain elements=2 {4,5})
- **Expected Output:** Injective, Surjective, Bijective
- **Actual Output:** This function is Injective, Surjective, Bijective.
- **Status:** Pass

11. Future Improvements

The improvements in future will be:

- **Performance Optimizations:** Improve efficiency for larger input values.
- **Feature Extensions:** Add graphical visualization of functions in a colorful format.
- **Scalability Improvements:** Support dynamic input sizes.
- **User Experience Enhancements:** Improve CLI with clearer prompts and also colorized this. with CLI and also GUI command line interface.
- **Algorithm Alternatives:** Explore stronger cryptographic techniques and also adding remaining cryptographic techniques.
- **Real-World Applications:** Extend for secure data validation systems and also adding others menu for more comprehensive view and helps in solving most difficult problems.

12. Conclusion

- **Key Achievements:** In this project, we successfully developed a Function Properties Checker using C++ programming language. The program correctly checks whether a function is injective, surjective, or bijective. We also implemented the inverse of a function and composition of functions, which helped us understand how functions interact with each other. We also added a basic cryptography feature that allows message encryption and decryption. We learned how functions are implemented in the code and how it works.

- **Learning Outcomes:** *This project highly improved our understanding of Discrete structures concepts, especially functions, relations, mappings, inverse and composition of functions. Concepts that were studied in old schools became much clearer when we applied them in C++. The project also strengthened our logical thinking and problem-solving skills.*
- **Challenges Overcome:** During development, we faced several challenges, particularly in applying the correct conditions for injectivity, surjectivity, and bijectivity. Handling incorrect inputs and ensuring accurate function mappings required strong concepts of functions and careful thinkings. Implementing inverse functions was also challenging, as it required strict checking of domain and range values. In cryptography, we also facing challenges when we encrypt or decrypt data.
- **Reflection:** This project was very significant for us because it helped us understand how theoretical concepts are applied in practical programming. Before working on this project, ideas like injective, surjective, and bijective functions were mostly mathematical definitions. Implementing them in C++ showed us how these concepts work in real-world problems and how good logic is required to get better results.
- **Final Remarks:** This project clearly demonstrates the strong connection between theoretical concepts and practical implementation by implementing concepts of Discrete structures in C++. By implementing function properties, inverse functions, composition, and basic cryptography, we learned that even small theoretical mistakes can lead to incorrect behavior of program. As I conclude this project, Discrete Mathematics has various application in programming and use of discrete structures are very vast in programming languages and plays a central role in solving real-world problems.

13. References & Learning Sources

- **GitHub:** [Click here to access code of DM-Final-Project](#) uploaded in GitHub.
- **Video Demo:** Link to a demonstration video, preferably uploaded on LinkedIn. [Click Here to view](#)
- **Textbooks:** Rosen, K. H. **Discrete Mathematics and Its Applications 8th edition.** Chapter(s):2,4.
- **Online Documentation:** Wikipedia (<https://www.wikipedia.org/>) and Paul's Online math notes (<https://www.onlinemathnotes.org/>)
- **Tutorials & Guides:** No Video Tutorials
- **Academic Papers:** No academic papers were read.
- **Code References:** We used GeeksforGeeks website for learning syntax of programming language C++.
- **AI Assistance:** No use of any AI source for help or assistance.